

Grow and Refine — Talk Script

Target: ~15 minutes. Casual, lab-meeting style. Each slide ~60-90s.

Framing rule (updated): Slides 1-6 stay GENERIC — "long-horizon engineering agents." We do NOT mention machine-learning engineering, Kaggle, or any specific domain until Slide 7 (Results: Setup), where we narrow down to MLE as the concrete instance we evaluate on.

Slide 1 — Title (~15 s)

Hi everyone. Today I want to walk you through *Grow and Refine* — our work on giving long-horizon engineering agents a memory that actually keeps getting better with experience, instead of just getting bigger.

Slide 2 — Engineering Task (~80 s)

Let me first set up what kind of task we care about. We're looking at **long-horizon engineering tasks** more broadly — the kind of work where an agent has to think, write code, run it, get feedback, and keep iterating until something works. Two things make this kind of task interesting for memory design.

First — it's long horizon, with dense feedback. The agent runs for ten or more turns. Each turn has some reasoning plus a code snippet, and after running the code, the agent gets real feedback — a test result, an error trace, a metric, whatever. So there's a constant stream of experience flowing in. The natural question is: **how does the agent use all this experience?** In memory terms, *how to use memory?*

Second — engineering work is a bag of tricks domain. Human experts beat top agents not because humans are smarter — it's because experts have accumulated a huge number of small, situation-specific tricks: which design pattern fits which kind of failure, etc. on the other side, Agents only know a handful of general methods. So if we want to close the gap, the agent has to keep **accumulating** tricks over time. In memory terms, *how to grow memory?*

These two questions — *how to use*, *how to grow* — drive everything in the rest of the talk.

Slide 3 — Motivation: flywheel, not pile (~90 s)

OK so once we agree memory is the lever, the next question is *what kind of memory*. Our claim is: memory should behave like a **flywheel** — it should gain momentum as the agent gets more experience — not like a pile that just gets bigger and messier.

Three design axes here.

A — How to use memory. Most existing work injects the whole memory in one big block at the start of a trajectory. That's static; the agent can't reconsider per action. We do it **turn by turn**, conditioned on what action the agent is taking right now.

B — How to grow memory. Existing work either just appends every new lesson (unbounded, contradictions pile up) or rewrites entries in place (loses past wisdom). We do **dual update**: memory both grows — we append new validated insights, that's the Δ -append part — AND self-corrects — we attach scoped caveats to entries that were over-generalized.

C — Who evolves memory. A large amount of prior work is single-agent: one model reads and writes its own memory, so the memory is bounded by that one model's mistakes and strengths. We do **multi-agent**: a single shared pool that multiple backbones read from and write into. So insights complement each other instead of being redundant — one model's blind spot is another model's strength.

Three axes: turn-level read, dual update, multi-agent. That's what we mean by *flywheel*.

Slide 4 — Per-turn, action-conditioned retrieval (~75 s)

Let me get into the method. First piece: how we use memory.

The agent does one of three things every turn — it either **drafts** a new candidate solution, **improves** the current one, or **debugs** a broken one.

Two consequences. On the **write side**: we only write a memory entry if the turn was actually successful — a valid draft, an improve that genuinely raised the score, or a debug that actually fixed the bug. Failed exploration just doesn't enter memory. The pool stays clean.

On the **read side**: when the agent is about to draft, it retrieves only past successful drafts. When it's about to improve, only past improves. Debug to debug. Each action gets advice from exactly its own kind of past experience.

(Note: there's still a control we want to run that compares this action-split retrieval against a unified retrieval — that'll confirm the separation is the thing that matters, not just having retrieval at all.)

Slide 5 — Grow and Refine (~100 s)

This is the central method slide. After round t , the agent has used memory v_t . Now we want to produce v_{t+1} . We do two things in parallel.

First — Grow. We scan every trajectory from round t and we keep the wins: every valid draft, every improve turn that raised the score, every debug-fix. Each of those wins becomes a new entry in memory. The effect: memory **covers more of the strategy space**.

Second — Refine. We look at tasks where the agent did **worse** with memory than without it — those are regressions. for each regression, we go into the trajectory and **pinpoint the single culprit entry** — the one specific retrieved insight that the agent actually copied and over-applied. Then we attach a **scope caveat** to that one entry. A caveat is a structured warning that says "when condition X holds, do NOT apply this insight." The original insight stays in memory; only its applicability gets narrowed. So we don't lose past wisdom — we **refine its boundary**, surgically.

Together, Grow plus Refine produces v_{t+1} . The agent enters round $t+1$ with that, and the cycle repeats. That's the flywheel.

Slide 6 — Co-evolving memory across multiple agents (~60 s)

Quick slide on the multi-agent piece. The memory pool is one single store, but different model backbones all read from it and write into it.

Each round, every agent's wins go into the pool as new entries, and every agent's regressions trigger caveats. Memory at the next round is the **union** of all agents' updates. So one model's mistake becomes everyone's caveat, and one model's win becomes everyone's insight. We get to combine strengths across backbones, not be bottlenecked by any one model.

Slide 7 — Results: Setup (~75 s) (*← MLE narrowing happens here*)

OK, time to evaluate. So far everything we said is for long-horizon engineering broadly. **To run actual experiments we need a concrete domain, and we picked machine-learning engineering — specifically, Kaggle competitions.** Why this choice: it's long-horizon (each task has many turns of train-debug-improve), the agent gets dense scored feedback, and crucially the Kaggle public leaderboard gives us an external, gameable-proof metric.

Concretely:

- **Benchmark:** 18 real Kaggle competitions from a benchmark called **DSGym**.
- **Metric:** Kaggle public leaderboard percentile — externally validated, not something we can game internally.
- **Backbones:** three different model families — Claude Sonnet 4.6, GPT-5.2, Gemini 3 Flash.
- **Ablations:** we cover static memory (no flywheel), Δ -only update, caveat-only update, trajectory-level memory, single-model vs multi-model, on-the-fly update, comparison with an external baseline called ACE, and different retrievers. Green checks are what we already have results for.

Slide 8 — Main results (~90 s)

Headline table. Each row is a backbone. The first three rows are the baselines — just the backbone running by itself, no memory framework. Sonnet 4.6 gets 46.4 on easy and 35.3 on hard. GPT and Gemini are in the same range.

After Grow and Refine:

- **Sonnet 4.6:** 64.7 on easy, 52.6 on hard — that's plus 39 percent on easy and plus 49 percent on hard. It also earns medals on hard that the baseline did not get.
- **GPT-5.2:** plus 31 percent and plus 29 percent.
- **Gemini 3 Flash:** plus 3.5 percent on easy, plus 26 percent on hard.

So it's not a single-model fluke — **three different model families all consistently improve, and all of them earn medals on the harder split.**

Slide 9 — Memory update method (the figure) (~120 s)

Now zoom into the update-method ablation. Same Sonnet 4.6 backbone, same retrieval, same memory pool — only the update method differs. Four lines per subplot: ACE (an external baseline from Zhang et al.), **grow-only**, **refine-only**, and **dual update** — that's our method, the bold green one.

Start with the **easy split** on the left. All four curves start at 46.4 — that's the no-memory baseline at step 0. By step 2 you can already see dual update at 61.8, the other three trailing.

The interesting story is from **step 3 onwards**. Grow-only drops from 58.9 to 54.5 and ends at 55.9 — pure accumulation backfires because some accumulated insights are over-general and the agent over-applies them. Refine-only stalls around 56 to 57 — it can scope existing entries but it doesn't add new capabilities. Only **dual update keeps climbing**: 59.2, 63.4, 64.7.

Hard split on the right tells the same story — dual update ends at 52.6, others at 47 to 49.

Takeaway: **grow or refine alone is not enough. You actually need both — that's what makes the flywheel compound instead of stall.**

Slide 10 — Turn-level vs trajectory-level memory (~75 s)

Now the other axis: how memory is used. Trajectory-level means we inject everything in one block at the start of the task. Turn-level is what we proposed earlier — re-retrieve at each turn, conditioned on the current action.

On Sonnet 4.6: turn-level gives 48.8 on easy versus trajectory-level 45.3 — about 7.7 percent gain. On hard the gap is much larger: 42.2 versus 31.0 — that's plus 36 percent.

Same pattern on GPT-5.2: turn-level wins on both splits, and again the gap on hard is much bigger.

So **per-turn retrieval helps more on hard problems**. That makes sense: hard tasks have more turns, more decisions, more places where the right memory at the right moment matters. Trajectory-level memory just can't reactively give different advice at different turns.

Slide 11 — Group wisdom vs single-model knowledge (~60 s)

This is the multi-agent comparison. Two settings: every backbone keeps its own per-model memory ("single-model knowledge"), versus everyone reading and writing the same shared pool ("group wisdom"). The early signal is that the shared pool helps every individual model — because one model's regression becomes a caveat that protects the others, and one model's trick gets reused everywhere.

(Full table coming in the next iteration — we're still completing the comparison runs.)

Slide 12 — Retriever Sensitivity (~75 s)

Last ablation: does the retriever matter? Same memory pool, same dual update, Sonnet backbone, only the retriever changes.

- **No memory**: 46.4 / 35.3 — baseline.

- **BM25** (lexical, no embedding): 59.5 / 44.2.
- **text-embedding-3-small** (our default): 61.8 / 47.4.
- **text-embedding-3-large**: 63.1 / 48.9.
- **Qwen3-Embedding-8B** (open-source SOTA): 62.0 / 46.8.

Two takeaways. **First**, every retriever — even the simplest BM25 — already gets most of the lift. So the framework isn't picky about which retriever you plug in. **Second**, bigger embedders help, but the marginal gain is modest. Most of the improvement comes from the **memory design**, not from how sophisticated the retriever is. That's a good property: the result transfers to setups without access to OpenAI embedding APIs.

Closing (~30 s)

So to wrap up — Grow and Refine is a memory flywheel for long-horizon engineering agents. Three design choices: per-turn action-aware retrieval, dual update with append-AND-scope-caveat, and a multi-agent shared pool. On 18 Kaggle competitions across three model backbones, public leaderboard percentile goes up by 30 to 50 percent, and the wins come from both operators together — neither one alone is enough. Happy to take questions.

Timing budget recap

Slide	Time	Cumulative
1 Title	15s	0:15
2 Engineering Task	80s	1:35
3 Flywheel motivation	90s	3:05
4 Per-turn retrieval	75s	4:20
5 Grow and Refine	100s	6:00
6 Multi-agent	60s	7:00
7 Setup (MLE narrowing)	75s	8:15
8 Main results	90s	9:45
9 Update-method figure	120s	11:45
10 Turn vs trajectory	75s	13:00
11 Group wisdom	60s	14:00
12 Retriever sensitivity	75s	15:15
Closing	30s	15:45

Total target: ~15 min. With pauses, brief asides, and 1-2 questions mid-talk, this naturally lands between 14 and 17 minutes.